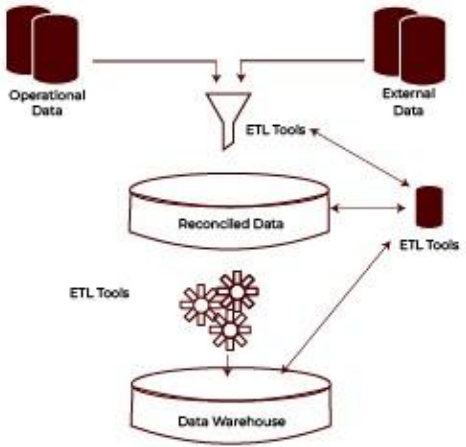
[DM] Project 2024/2025

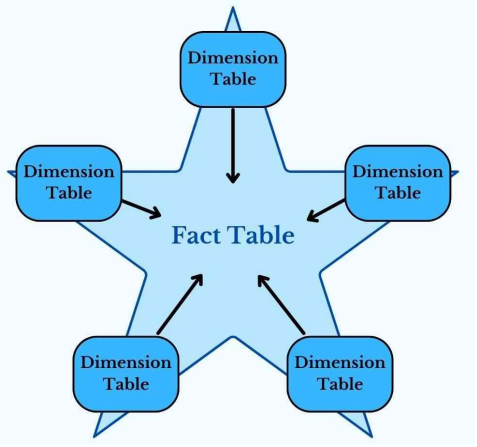
Football Matches Data Warehouse

Idea of the project

Type of project

Data warehouse: select a set of data sources, integrate them by means of ETL operations, define the DFM model for your data, define and populate the corresponding star schema or snowflake schema.





Theme

I decided to develop a project about **football matches** in the *top 5 European leagues*, played between seasons *2010/2011* and *2018/2019*.

For each match, I'm interested in the final score (represented as number of goals of both the home and the away team) and in the attendance.

Each match is characterized by other properties, like the referee, the date, the involved teams and other interesting facts about them.

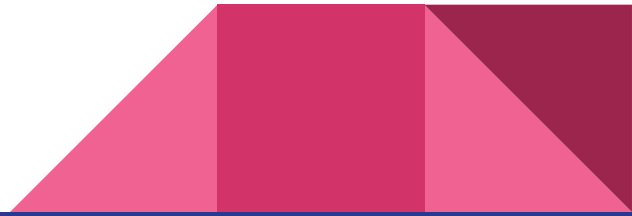
Layer 1 - Sources

European Football Games

This dataset contains *matches* from 2004/2005 to 2018/2019 between teams in Premier League, Serie A, Ligue 1, La Liga, Bundesliga.

For each match I'm interested in *Away Coach*, *Away Goals*, *Away Name*, *Date*, *Home Coach*, *Home Goals*, *Home Name*, *League*, *Referee*, *Season*, *Stadium*, *Visitor Count*.

Data from these attributes are the main one that will populate the data warehouse, describing each match in detail.



Football Data: Competitions, Clubs, Players

From this dataset I decided to take only the csv file describing all the European Football *competitions*.

I'm interested in *Name*, *Type*, *Country Name*.

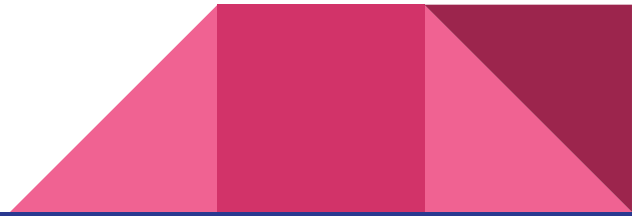
These data are useful to join on the League name with the one stored in the matches and to acquire the information of the type of competition in which the match has been played and the country to which that competition belongs.

Top 5 European football leagues teams stats

This dataset contains some *statistics* about the teams playing in the top 5 European Leagues, from season 2010/2011 up to season 2020/2021.

I’m interested in *Competition*, *Season*, *Rank*, *Squad*.

The idea is to ignore actual statistics, but to use this information to link to each team involved in a match the final position in its league in that particular season, as well as to link the actual winner of the competition to the league itself.



Football Stadiums

This is a list of football *stadiums* all across the world, containing their capacity and other information.

I'm interested in *Stadium*, *City*, *Capacity*, *Country*, *Population*.

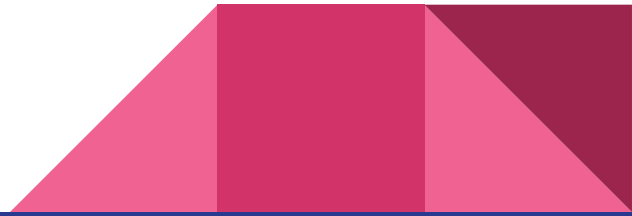
I need these data to integrate the stadium in which the match was played and the attendance to that game with the actual capacity of the stadium and the city in which it is located.
I also want to join the competitions on the country in which they are played to store the population of such countries.

European Football (soccer) Clubs on Google SERPs

This dataset records victories of teams in *European Tournaments*.

I'm interested in *Club*, *#UCL* (UEFA Champions League), *#UEL* (UEFA Europa League), *#CWC* (UEFA Cup Winners Cup), *#USC* (UEFA Super Cup).

These data are useful to associate the number of european tournaments wins to each team stored in the database (0 if the team does not appear in this list).



Layer 2 - Reconciled Data

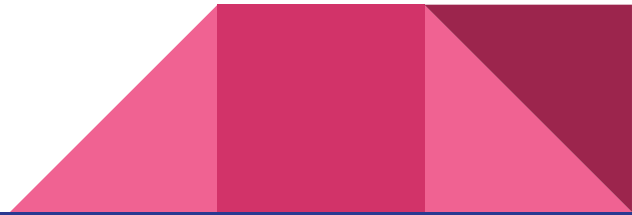
ETL Operations

ETL Operations

First of all, I loaded all the sources in python as pandas dataframes, then I **dropped** all the columns I was not interested in.

To help joins and to conform to common knowledge, I **replaced** some values with more appropriate ones (e.g. Primera División -> La Liga).

Later on in the code, I also created the columns ‘month’ and ‘year’, starting from the attribute ‘date’, which will be needed in the data warehouse.



Since the matches database refers to seasons 2004/2005 - 2018/2019, while the stats database refers to seasons 2010/2011 - 2020/2021, I decided to **filter** their rows to keep only the ones about seasons in the intersection.
The result consists in two datasets referring to seasons **2010/2011 - 2018/2019**.

The two datasets have also the season formatted in different ways, so I needed to extract the starting year of the season through regular expressions, and to uniform the format at the end (season yyyy/yyyy).

```
# Take the starting year of the season for each row and store it to use it as a filter.
df_matches['season_start'] = df_matches['season'].str.extract(r'(\d{4})').astype(int)
df_matches = df_matches[(df_matches['season_start'] >= 2010) & (df_matches['season_start'] <= 2018)]
df_matches.drop(columns='season_start', inplace=True)
# Do the same thing for the stats dataframe.
df_stats['season_start'] = df_stats['season'].str.extract(r'(\d{4})').astype(int)
df_stats = df_stats[(df_stats['season_start'] >= 2010) & (df_stats['season_start'] <= 2018)]
df_stats.drop(columns='season_start', inplace=True)
# Problem: df_matches has the year written as '2010/2011', while df_stats uses a format like '2010-2011'.
# Since I will need to join on the season, I need to uniform the format of the two dataframes,
# and I will do that by transforming the hyphen (-) into a slash (/).
df_stats['season'] = df_stats['season'].str.replace('-', '/')
```

Join Matches and Leagues

Fuzzy matching: I wanted to join matches dataframe with leagues one on the competition of the match to store the type and the country of the competition in which the match was played.
However, names of the leagues are slightly different in the two databases, so I needed to compute the similarity between them, to be able to join in presence of high similarity scores (> 80), instead of only looking for exact matches.

```
# Store the names of the leagues of the competitions database.
league_names = df_leagues['name'].unique()
# Define a function that, given a league (from matches), checks for the most similar
# league in the stored ones and returns the most accurate match only if it has a score > 80.
def getLeague(league):
    match, score, _ = process.extractOne(league, league_names)
    return match if score > 80 else None
# Create a new column in the matches database with the matched league.
df_matches['matched_league'] = df_matches['league'].str.lower().apply(getLeague)

# Merge using the normalized matched league name, then remove the useless columns.
df1 = df_matches.merge(df_leagues, left_on='matched_league', right_on='name', how='left')
df1.drop(columns=['matched_league', 'name'], inplace=True)
```

Join with Stadiums

Using stadiums dataframe, I performed **two joins**:
- one on the stadium itself, to store its capacity and the city in which it is (and thus in which the game has been played);
- the other one on the country, to store its population.

The problem is that there are some different stadiums with the same name, so I needed to perform a deeper check: I **filtered** the stadiums in such a way to have only rows belonging to the top 5 countries (and then leagues), then I **sorted** the stadiums on [name, population], keeping only the first one in case of **duplicates**.

However, this was not enough. Even solving the duplicates problems, there were mismatches due to the fuzzy matching joining not correct pairs.
Tuning the similarity score required to perform the join was not the right solution, because lowering it was introducing even more mistakes, while increasing it was causing the lost of some correct matches.
I decided to **nest** another **fuzzy matching** but on the city on each pair of matched stadium above a certain threshold, to add a second level of verification.

```
stadium = row['stadium']
location = row['location']
if pd.isnull(stadium) or pd.isnull(location): return None
match, score, _ = process.extractOne(stadium, stadium_names, scorer = fuzz.partial_ratio)
if score < 65: return None
# If the score is greater compare also the city.
matched_row = df_stadiums_unique[df_stadiums_unique['Stadium'] == match]
matched_city = matched_row.iloc[0]['City']
city_score = fuzz.partial_ratio(str(location).lower(), str(matched_city).lower())
return match if city_score > 80 else None
```

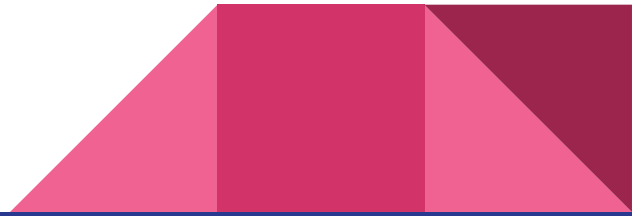
Join with Stats

In this case, I wanted to join on the competition, the season and the team, to add to the dataframe the final position of such team in that competition in that season. As in the previous case, there were some problems with the matchings, this time about teams. I needed to **replace** the value of some teams to be able to find them in the fuzzy matching (e.g. Queens Park Rangers -> QPR, …), but also to **check** the league above certain similarities scores to make a decision about matchings. Since some teams have similar names even if they are different, I wanted to be sure that matched teams come from the same league.

Join with Trophies

To add the number of trophies won by each team I just needed to perform an additional **fuzzy matching** equivalent to the previous ones, both on the home team and on the away team, followed by a **fillna** to assign the value ‘0’ to each team absent in the trophies dataset.

However, also in this case I needed to perform some **manipulation** to help the fuzzy matching, because some values were written in a format that was making the matching impossible (e.g. M'Gladbach -> Borussia Mönchengladbach, …).



Relational Database

Reconciled Data Layer on PostgreSQL

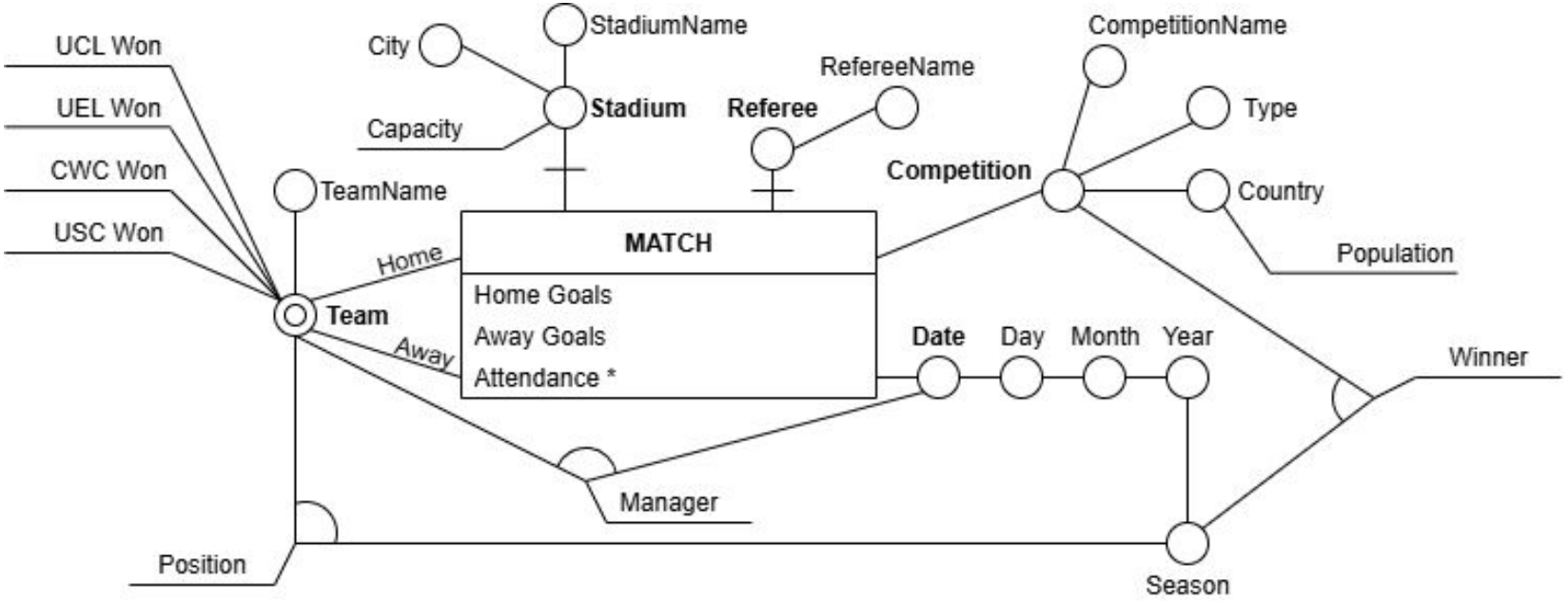
After I solved all the ETL issues, I decided to load the unified dataframe, representing the reconciled layer, in a relational database.
I chose PostgreSQL to do it; at the beginning of the script I established a connection with the pre-existing dedicated database and, after the cleaning of the data, I created a table ‘football_matches’ in such database containing the reconciled database.

```
# Create SQLAlchemy engine to interact with PostgreSQL.
engine = create_engine(f'postgresql+psycopg2://{user}:{password}@{host}:{port}/{dbname}')
# Write the dataframe to PostgreSQL (replace table if it exists).
rdl.to_sql(
    name='football_matches',
    con=engine,
    if_exists='replace',
    index=False
)
print("✅ Data successfully loaded into table 'football_matches'!")
```

Layer 3 - Data Warehouse

Dimensional Fact Model

DFM


UCL Won
UEL Won
CWC Won
USC Won
City
StadiumName
Capacity
Stadium
RefereeName
Referee
CompetitionName
Type
Competition
Country
Population
TeamName
Home
MATCH
Home Goals
Away Goals
Attendance *
Team
Away
Date
Day
Month
Year
Winner
Manager
Position
Season

About the DFM

Conceptual model of the data warehouse:

Fact (event I'm interested in) -> *Match*;

Measures (numerical attributes quantifying the fact) -> *Home Goals*, *Away Goals*, *Attendance* * (not mandatory);

Dimensions (main properties of the fact) -> *Home Team*, *Away Team*, *Competition*, *Date*, *Referee* *, *Stadium* *;

Dimensions are associated with hierarchies of attributes, some of which are descriptive (not useful for aggregation).

Facts are identified by the subset of mandatory dimensions [*Home Team*, *Away Team*, *Competition*, *Date*].

I used a shared hierarchy on *Team* to distinguish between the home and the away one without replicating the same attributes.

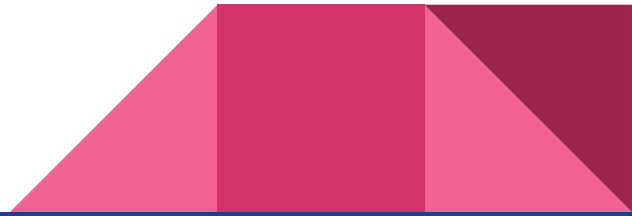
There are three cross-dimensional attributes, which are *Position* and *Winner*, because both attributes' values depend on the analyzed season, and *Manager*, which depends on the date because a team may change manager also multiple times during a single season.

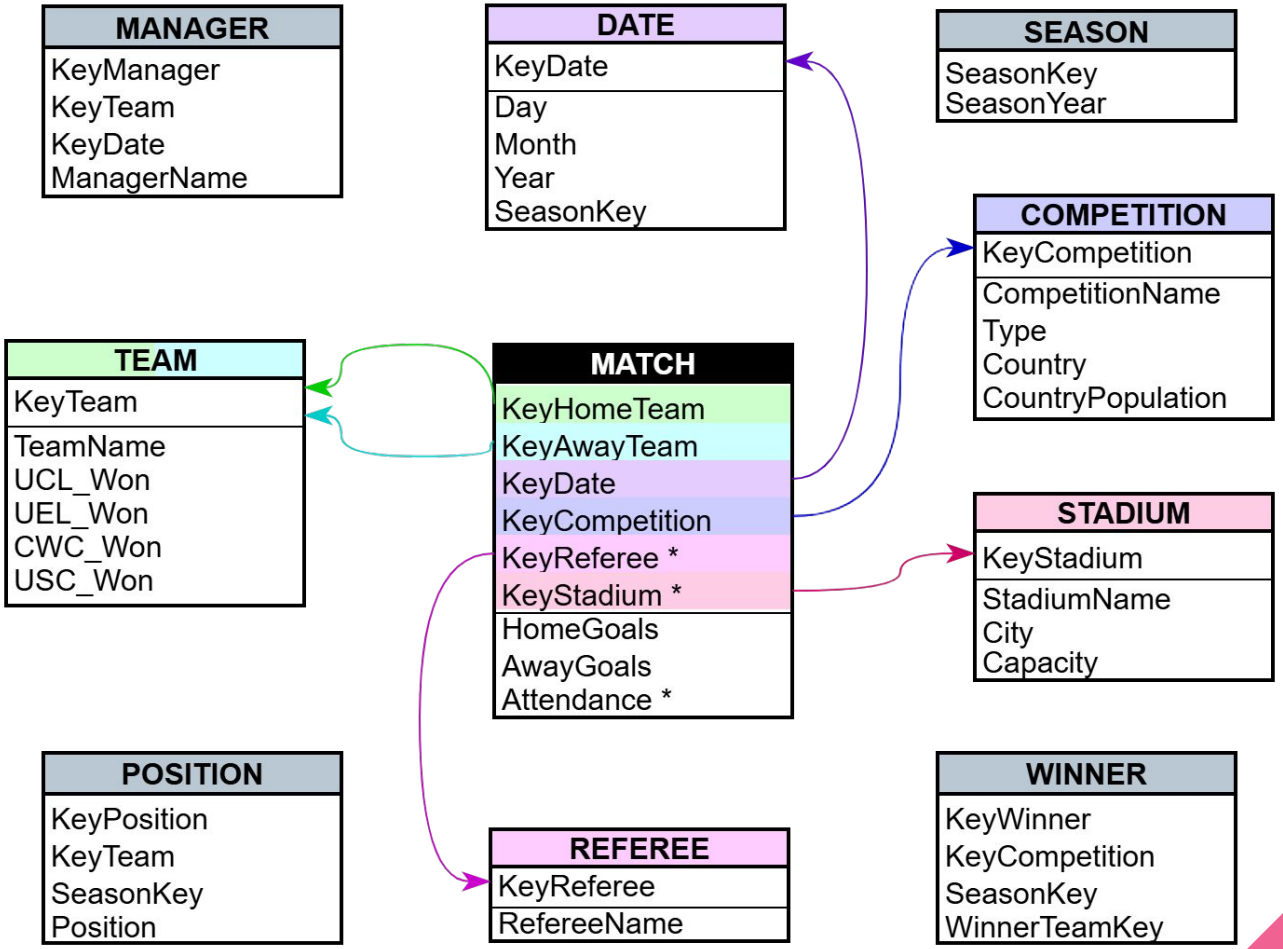
Star Schema

Logical Model - ROLAP: Star Schema

I chose **star schema** because I think football analysis, in particular when it comes to matches with a lot of parties involved, requires many interactions between the sources. In this sense the star schema is more efficient, because it allows to perform just few joins to retrieve information at the cost of redundancy.

Star schema has a central fact table, linked to all the dimension tables through surrogate keys. However, I need to take care of cross-dimensional attributes (Winner and Position). I will create additional tables for them.





The translation is almost 1:1 from the DFM.
I uniformed both the home and the away teams in a single table, and I needed to create a table for the seasons to reference the key for cross dimensional attributes.

This way of treating seasons is typical of a **snowflake**, but I think it is not enough to move away from the star schema, since everything else is heavily relying on efficient joins and denormalization.

MATCH (key_home, key_away, key_date, key_competition, key_referee *, key_stadium *, home_goals, away_goals, attendance *)

TEAM (key_team, team_name, ucl_won, uel_won, cwc_won, usc_won)

DATE (key_date, day, month, year, key_season)

COMPETITION (key_competition, competition_name, type, country, country_population)

REFEREE (key_referee, referee_name)

STADIUM (key_stadium, stadium_name, city, capacity)

MANAGER (key_manager, key_team, key_date, manager_name)

POSITION (key_position, key_team, key_season, position)

WINNER (key_winner, key_competition, key_season, key_winner_team)

SEASON (key_season, season_year)

Data Warehouse on PostgreSQL

DW

To create and populate the star schema in PostgreSQL, I defined two connections, one to the Reconciled Data Layer database to download the unified table, and one to the Data Warehouse database to load the data.

First of all, I casted all the numerical columns values to Int64, to be sure to work with integers while preserving null values (which I replaced with None).

For each dimension, I stored its content by taking the desired columns from the reconciled data layer, taking care of renaming them according to the star schema and creating its surrogate key using the index of the dataframe.

As for the fact table, I stored its content by taking the original reconciled data layer and by merging it with each defined dimension to store the respective surrogate keys, then I only kept those columns and the ones referring to the measures.

Then, I used the python library **sqlalchemy** to define the schema of the tables with columns, types, and primary and foreign keys, before inserting the stored data into them using the connection with the database.

```
match_fact = Table('match', metadata,
    Column('key_home', ForeignKey('team.key_team'), Integer),
    Column('key_away', ForeignKey('team.key_team'), Integer),
    Column('key_date', ForeignKey('date.key_date'), Integer),
    Column('key_competition', ForeignKey('competition.key_competition'), Integer),
    Column('key_referee', Integer, ForeignKey('referee.key_referee'), nullable=True),
    Column('key_stadium', Integer, ForeignKey('stadium.key_stadium'), nullable=True),
    Column('home_goals', Integer),
    Column('away_goals', Integer),
    Column('attendance', Integer, nullable=True),
    PrimaryKeyConstraint('key_home', 'key_away', 'key_date', 'key_competition')
)
```

Example of usage

Query 1 - Basic Cube

Recompute the basic cube.

```
SELECT  x1.manager_name as "Home Coach", t1.team_name as "Home Team", m.home_goals as "Home Goals",
        x2.manager_name as "Away Coach", t2.team_name as "Away Team", m.away_goals as "Away Goals",
        d.day as "Date", c.competition_name as "League", c.type as "Type",
        r.referee_name as "Referee", y.season_year as "Season", m.attendance as "Visitor Count",
        c.country as "Country", c.country_population as "Population",
        s.stadium_name as "Stadium", s.city as "City", s.capacity as "Capacity",
        p1.position as "Home Rank", p2.position as "Away Rank", t3.team_name as "League Winner",
        t1.ucl_won as "UCL Home", t1.uel_won as "UEL Home", t1.cwc_won as "CWC Home", t1.usc_won as "USC Home",
        t2.ucl_won as "UCL Away", t2.uel_won as "UEL Away", t2.cwc_won as "CWC Away", t2.usc_won as "USC Away"
FROM    match m
        JOIN team t1 ON m.key_home = t1.key_team
        JOIN team t2 ON m.key_away = t2.key_team
        JOIN date d ON m.key_date = d.key_date
        JOIN competition c ON m.key_competition = c.key_competition
        LEFT JOIN referee r ON m.key_referee = r.key_referee
        LEFT JOIN stadium s ON m.key_stadium = s.key_stadium
        JOIN season y ON d.key_season = y.key_season
        JOIN manager x1 ON (d.key_date = x1.key_date AND t1.key_team = x1.key_team)
        JOIN manager x2 ON (d.key_date = x2.key_date AND t2.key_team = x2.key_team)
        JOIN position p1 ON (y.key_season = p1.key_season AND t1.key_team = p1.key_team)
        JOIN position p2 ON (y.key_season = p2.key_season AND t2.key_team = p2.key_team)
        JOIN winner w ON (c.key_competition = w.key_competition AND y.key_season = w.key_season)
        JOIN team t3 ON t3.key_team = w.key_winner_team
```

Query 2 - OLAP operators

ROLL UP: Aggregate data up in the hierarchy.

```
SELECT      t.team_name,
            d.day,
            c.competition_name,
            m.attendance
FROM        match m
            JOIN team t ON m.key_home = t.key_team
            JOIN date d ON m.key_date = d.key_date
            JOIN competition c ON m.key_competition = c.key_competition
```

	team_name character varying (100)	day date	competition_name character varying (100)	attendance integer
1	Hércules	2010-08-28	La Liga	28000
2	Málaga	2010-08-28	La Liga	19400
3	Levante	2010-08-28	La Liga	15200
4	Espanyol	2010-08-29	La Liga	27000

```
SELECT      y.season_year,
            t.team_name,
            c.competition_name,
            SUM(m.attendance) as "Total Attendance"
FROM        match m
            JOIN team t ON m.key_home = t.key_team
            JOIN date d ON m.key_date = d.key_date
            JOIN competition c ON m.key_competition = c.key_competition
            JOIN season y ON d.key_season = y.key_season
GROUP BY    y.season_year, t.team_name, c.competition_name
ORDER BY    y.season_year, c.competition_Name, "Total Attendance" DESC
```

	season_year character varying (20)	team_name character varying (100)	competition_name character varying (100)	Total Attendance bigint
1	2010/2011	Dortmund	Bundesliga	1351600
2	2010/2011	Bayern Munich	Bundesliga	1173000
3	2010/2011	Schalke 04	Bundesliga	1042000

On the left I executed a basic query to retrieve the attendance of each match, projecting only the home team which is the one that hosts the game. On the right instead, I **aggregated** the **visitor count** measure on the **time hierarchy**, showing the value over the entire season. To do so, I needed to aggregate also on the team and the competition, otherwise the result wouldn't have been meaningful.

DRILL DOWN: Reduce the level of aggregation.

```
SELECT      d.month,
            t.team_name,
            c.competition_name,
            SUM(m.attendance) as "Monthly Attendance"
FROM        match m
            JOIN team t ON m.key_home = t.key_team
            JOIN date d ON m.key_date = d.key_date
            JOIN competition c ON m.key_competition = c.key_competition
GROUP BY    d.month, t.team_name, c.competition_name
ORDER BY    d.month, c.competition_Name, "Monthly Attendance" DESC
```

	month character varying (20)	team_name character varying (100)	competition_name character varying (100)	Monthly Attendance bigint
1	2010-08	Dortmund	Bundesliga	73300
2	2010-08	Bayern Munich	Bundesliga	69000
3	2010-08	Schalke 04	Bundesliga	61300
4	2010-08	Hamburger SV	Bundesliga	57000

Starting from the previous aggregation level on the season, I went back to a more detailed level of aggregation considering the **month**.

SLICE: Filter on one dimension fixing a specific value.
and *DICE*: Reduce the set of data by fixing a range of possible values.

```sql
SELECT      y.season_year as "Season",
            t1.team_name as "Home Team",
            t2.team_name as "Away Team",
            c.competition_name as "League",
            m.home_goals as "Home Goals",
            m.away_goals as "Away Goals",
            m.attendance as "Attendance"
FROM        match m
            JOIN team t1 ON t1.key_team = m.key_home
            JOIN team t2 ON t2.key_team = m.key_away
            JOIN date d ON d.key_date = m.key_date
            JOIN season y ON y.key_season = d.key_season
            JOIN competition c ON c.key_competition = m.key_competition
WHERE       c.competition_name = 'Serie A' AND
            m.attendance > 30.000 AND
            (m.home_goals + m.away_goals) >= 2
ORDER BY    season_year, "Attendance" DESC
```

Data Output | Messages | Notifications

Showing rows: 1 to 1000 | Page No: 1

	Season character varying (20)	Home Team character varying (100)	Away Team character varying (100)	League character varying (100)	Home Goals integer	Away Goals integer	Attendance integer
1	2010/2011	Milan	Juventus	Serie A	1	2	80000
2	2010/2011	Milan	Inter	Serie A	3	0	79000

To use this operator I sliced on the competition, fixing it only to ‘Serie A’, then I performed the dice operation on the attendance and the total number of goals of the matches.

Query 3

Find all the matches won or drew by West Ham in 2018 (return the name of the teams, the competition, the score and the date).

Query | Query History

```
1  SELECT  t1.team_name as "Home", t2.team_name as "Away",
2          c.competition_name, m.home_goals, m.away_goals,
3          d.day
4  FROM    match m JOIN team t1 ON m.key_home = t1.key_team
5          JOIN team t2 ON m.key_away = t2.key_team
6          JOIN competition c ON m.key_competition = c.key_competition
7          JOIN date d ON m.key_date = d.key_date
8  WHERE   ((t1.team_name = 'West Ham' AND m.home_goals >= m.away_goals) OR
9          (t2.team_name = 'West Ham' AND m.away_goals >= m.home_goals))
10         AND d.year = 2018
```

	Home character varying (100)	Away character varying (100)	competition_name character varying (100)	home_goals integer	away_goals integer	day date
1	Tottenham	West Ham	Premier League	1	1	2018-01-04
2	West Ham	West Brom	Premier League	2	1	2018-01-02

Query 4

Which stadiums had the highest annual average attendance in Ligue 1 matches after 2015?

```sql
SELECT      s.stadium_name AS "Stadium",
            s.city AS "City",
            ROUND(AVG(m.attendance)) AS "Avg Attendance",
            y.season_year AS "Year",
            COUNT(*) AS "# Matches"
FROM        match m
            JOIN competition c ON m.key_competition = c.key_competition
            JOIN date d ON m.key_date = d.key_date
            JOIN stadium s ON m.key_stadium = s.key_stadium
            JOIN season y ON d.key_season = y.key_season
WHERE       c.competition_name = 'Ligue 1' AND
            d.year >= 2015
GROUP BY    s.stadium_name, s.city, y.season_year
HAVING      COUNT(*) > 10
ORDER BY    "Avg Attendance" DESC
```

	Stadium character varying (100)	City character varying (100)	Avg Attendance numeric	Year character varying (20)	# Matches bigint
1	Orange Vélodrome	Marseille	52888	2018/2019	18
2	Parc des Princes	Paris	47070	2018/2019	18
3	Parc des Princes	Paris	46884	2017/2018	19
4	Orange Vélodrome	Marseille	46616	2017/2018	19
5	Groupama Stadium	Lyon	46535	2018/2019	18
6	Parc des Princes	Paris	45159	2016/2017	19

Query 5

Find referees who officiated more than 10 Serie A matches, and compute average goals per match.

```
SELECT      r.referee_name AS "Referee",
            COUNT(*) AS "# Matches",
            ROUND(AVG(m.home_goals + m.away_goals), 2) AS "Avg Goals"
FROM        match m
            JOIN referee r ON m.key_referee = r.key_referee
            JOIN competition c ON m.key_competition = c.key_competition
WHERE       c.competition_name = 'Serie A'
GROUP BY    r.referee_name
HAVING      COUNT(*) > 10
ORDER BY    "Avg Goals" DESC
```

	Referee character varying (100)	# Matches bigint	Avg Goals numeric
1	Daniele Chiffi	20	3.40
2	Christian Brighi	35	3.31

Data Management - 2024/2025

https://github.com/Ale-ssio/DM_Project.git

Data Warehouses Project

Vernarelli Alessio 1946128